



Les cookies et les sessions



The **Apache Software Foundation**
<http://www.apache.org/>



Cookies (1/5)



- ☐ HTTP est un protocole sans état (stateless)
Il est donc difficile de savoir si 2 requêtes proviennent du même client, ainsi que de suivre la navigation d'un client
- ☐ Une solution : les cookies
 - permettent de stocker des informations du côté du client
 - ces informations peuvent par la suite être utilisées pour fédérer toutes les requêtes provenant d'un même client
- ☐ Un cookie est une variable permettant de stocker un état (une valeur) pouvant être utilisé par la suite (même session ou lors d'une connexion ultérieure)
- ☐ Les cookies sont stockés côté client, et sont automatiquement accessibles côté serveur (uniquement par le serveur qui les a créés) lors d'une connexion



Cookies (2/5)



- ❑ Quelques restrictions côté client (navigateurs) selon les spécifications
 - 300 cookies au maximum par navigateur
 - 20 cookies au maximum dédiées à un serveur donné
 - la taille d'un cookie est limitée à 4 Ko
 - les cookies peuvent être refusés par un client
 - les cookies sont stockés dans un fichier texte côté client, et donc être « visités » sur la machine du client
- ❑ Une seule fonction pour effectuer toutes les opérations sur les cookies : `setcookie(...)`
- ❑ L'appel à `setcookie()` doit **impérativement** être réalisé **avant** les balises `<html>` ou `<head>`. En effet, les cookies sont transmis dans l'en-tête HTTP; Si ce n'est pas le cas, l'appel ne fonctionne pas et renvoie FALSE. Si l'appel fonctionne, TRUE est renvoyé
- ❑ Il n'est pas possible de savoir si le client a accepté le cookie...



Cookies (3/5)



```
boolean setcookie(string nom,  
string valeur,  
int expire,  
string chemin,  
string domaine,  
int securise)
```

Tous les paramètres, sauf le premier, sont optionnels (utiliser la chaîne vide "" pour ignorer un paramètre, sauf expire et securise, utiliser 0)

nom : nom du cookie (si c'est le seul paramètre, cela entraîne sa suppression)

valeur : chaîne associée au cookie (accessible sous PHP dans \$nom)

expire : date et heure d'expiration (en s)

chemin : chemins du serveur où le cookie est valide

domaine : domaines sur lequel le cookie est valide

securise : si différent de 0, empêche l'envoi du cookie si la transmission n'est pas sécurisée



Cookies (4/5)



- ❑ Un cookie 'nom' est accessible en PHP
 - via `$_COOKIE["nom"]`
- ❑ Si un cookie n'existe pas, la variable correspondante n'existe pas (tester avec `isset()`)
- ❑ Les cookies sont persistants (perdurent même si le client s'arrête)
- ❑ Cadre typique d'utilisation des cookies : stockage d'un identifiant d'utilisateur (gestion de session)
- ❑ Les cookies de durée de vie -1 sont stockées uniquement en mémoire, ils sont donc non persistants après la fermeture du client (cookies de session)



Cookies (5/5)



```
<? // cookie pour rappeler la date de la visite précédente du client
// on récupère la valeur du cookie
isset($_COOKIE["datedepassage"]) ? $valcook=$_COOKIE["datedepassage"] : $valcook="";
// on met à jour le cookie chez le client
setcookie("datedepassage", date('d/m/Y \à H\h i\m\i\n s\s\c\c'), time()+20*24*3600);
?>

<html><head><title>Démo de cookie</title></head>
<body bgcolor="white">
<H1> Aujourd'hui nous sommes le
<? print(date('d/m/Y \à H\h i\m\i\n s\s\c\c')."<BR>et ");
if (empty($valcook)) { // le cookie n'était pas encore défini
    print ("c'est votre première visite chez nous"); }
else {
    print ("vous êtes déjà venu chez nous le : $valcook"); }
?>

</H1><BR><A HREF="ex.php">retour</A>
</body>
</html>
```



La fonction header()



- ❑ La fonction header() permet d'envoyer des en-têtes HTTP et doit, comme setcookie(), être placée avant la première balise HTML
- ❑ Documentation complète sur <http://www.php.net/manual/en/function.header.php>
- ❑ Une en-tête particulière : Location Permet de rediriger le client vers une autre adresse (Ex. header("Location: http://stri.net");)
- ❑ Pour vérifier qu'un utilisateur est passé par une page donnée

```
<?php
if (!isset($cookieVerif)) {
    header("Location: http://...");
    exit();
}
?>
```

Très pratique pour empêcher l'accès de certaines pages si un utilisateur n'est pas au préalable passé par d'autres pages...



Les sessions (1/3)



- ❑ Une session correspond à la durée de vie d'un navigateur visitant un site Web. En interne, PHP associe un identificateur unique à toute session (calculé en partie de façon aléatoire)
- ❑ Les sessions permettent ensuite de manipuler des variables de session \$_SESSION["nomVar"]. Ces variables sont accessibles dans **toutes** les pages utilisant les sessions PHP. Elles sont stockées habituellement dans des fichiers temporaires, côté serveur. Elles se manipulent comme les autres variables, mais doivent être déclarées à l'aide de fonctions PHP
- ❑ Pour entrer dans le mode de suivi de session sous PHP, il est nécessaire de faire un appel à la fonction **session_start()**
 - dans chaque script PHP où cette gestion de session est nécessaire
 - avant toute balise HTML
- ❑ Cette fonction
 - crée une nouvelle session si aucune n'existe
 - récupère la session existante sinon



Les sessions (2/3)



- Manipulation des variables de session :
 - déclaration d'une variable de session `$_SESSION["var"] = ...`
 - suppression d'une variable de session `unset($_SESSION["var"])`
 - test pour savoir si une variable est déclarée `isset($_SESSION["var"])`

- La gestion de session peut être utilisée
 - directement, pour suivre un visiteur, qui ne possède pas de compte sur le site visité
 - après connexion d'un utilisateur, ce qui suppose qu'on procède à une authentification avant d'ouvrir une session

Dans le dernier cas, il faut évidemment gérer beaucoup plus d'informations du côté du serveur, et mettre en œuvre un SGBD par exemple...



Les sessions (3/3)



```
<?php
session_start();
$_SESSION["login"]="toto";
?>
```

```
<?php
session_start();
if (isset($_SESSION["login"]))
    echo "bonjour " . $_SESSION["login"];
else
    echo "Vous devez d'abord vous connecter";
?>
```